11

© 한상훈 NestJS 백엔드 마스터

NestJS 백엔드 마스터

PART 1

에러 응답 규격과 Problem Details

제1부 기반: 백엔드와 Node.js 런타임 · 3장 HTTP 기초

문장을 읽지 않고 다음 행동을 정하는 에러 본문



© 한상훈 NestJS 백엔드 마스터

409가 왔을 때

```
HTTP/1.1 409 Conflict
{"message":"Seat A-3-15 is already held","error":"Conflict","statusCode":409}
```

→ 앱 개발자: 고객 화면에 무엇을 띄울지

→ "409가 오면, 어떤 409인지 뭘 보고 알아요?"

→ 지금 본문에서 실패 종류를 알 수 있는 곳: message 문장뿐

© 한상훈 NestJS 백엔드 마스터

409 하나로 모르는 것

409 Conflict의 이유	고객 앱이 할 일
이미 선점된 좌석	좌석 지도로, 다른 좌석 안내
1인 4매 한도 초과	이미 고른 좌석 정리 안내
이미 결제가 끝난 예매	예매 내역 화면으로

→ 상태 코드: 실패의 종류까지

→ 어떤 실패인지: 본문에서

© 한상훈 NestJS 백엔드 마스터

Nest 기본 에러 본문 세 가지

(없는 공연, 404)

```
{"message":"공연 99번이 없습니다","error":"Not Found","statusCode":404}
```

(검증 실패, 400, ValidationPipe)

```
{"message":["seat must be a string","count must not be less than 1",
 "count must be an integer number"],"error":"Bad Request","statusCode":400}
```

(처리하지 못한 예외, 500)

```
{"statusCode":500,"message":"Internal server error"}
```

데이터 블록: NestJS 12.2.0, class-validator 사용 (부록 시트) · 촬영 직전 확인

© 한상훈 NestJS 백엔드 마스터

앱 토스트에 찍히는 것

```
toast(body.message);
// 404: "공연 99번이 없습니다"
// 400: seat must be a string,count must not be less than 1,count must be an integer number

toast('' + { reason: 'SOLD_OUT' });
// [object Object]
```

→ 배열: 쉼표로 이어진 영어 문장

→ 객체: [object Object]

© 한상훈 NestJS 백엔드 마스터

필드마다 다른 타입

경우	message	error
`404`	문자열	있음
`400` 검증 실패	문자열 배열	있음
`500`	문자열	없음

→ 같은 필드 이름, 다른 타입

→ 앱 코드: 경우마다 `if`

→ 앱이 iOS, 안드로이드, 웹이면 같은 `if` 세 벌

© 한상훈 NestJS 백엔드 마스터

문구로 분기하는 고객 앱

```
if (res.status === 409 && body.message.includes('already held')) {
  showSeatMap('다른 고객이 먼저 선택한 좌석입니다');
} else {
  toast('잠시 후 다시 시도해 주세요');
}
```

→ 가정: 서버 개발자가 문구 수정 `'already held'` → `'was taken'`

→ 결과: 좌석 지도 대신 '잠시 후 다시 시도', 같은 좌석 재시도 반복

→ 사람이 읽는 문장을 프로그램이 읽은 결과

© 한상훈 NestJS 백엔드 마스터

회사마다 다른 에러 형식

서비스	실패 종류를 담는 필드
우리 예매 서버	`message` 문자열
결제사 A	`code: "CARD_DECLINED"`
알림 발송사 B	`errorCode: 4012`
사내 정산 서비스	`error.type`

→ 서비스마다 다른 해석 코드

→ 표의 결제사·발송사: 예시

© 한상훈 NestJS 백엔드 마스터

RFC 9457 Problem Details

2016

RFC 7807

2023

RFC 9457

→ HTTP API의 에러 본문 표준 형식

→ 미디어 타입: `application/problem+json`

→ 2016년 RFC 7807 → 2023년 RFC 9457로 대체

→ 정해진 필드 다섯 개 + 서비스가 정하는 확장 필드

데이터 블록: RFC 9457 (부록 시트) · 촬영 직전 확인

© 한상훈 NestJS 백엔드 마스터

다섯 필드

필드	내용	누가 읽나
`type`	문제 종류를 가리키는 URI	프로그램
`title`	종류의 짧은 이름, 같은 type이면 같은 문장	사람
`status`	상태 코드 사본, 실제 상태 코드와 같은 값	프로그램
`detail`	이번 건의 설명	사람
`instance`	이번 건을 가리키는 URI	사람, 로그 검색

© 한상훈 NestJS 백엔드 마스터

이미 선점된 좌석

```
HTTP/1.1 409 Conflict
Content-Type: application/problem+json; charset=utf-8

{
  "type": "https://api.ticketing.example/problems/seat-already-held",
  "title": "이미 선점된 좌석",
  "status": 409,
  "detail": "A-3-15 좌석은 다른 고객이 선점했습니다.",
  "seat": "A-3-15",
  "instance": "/performances/1/holds"
}
```

→ `seat`: 확장 필드, 이 type이 정한 추가 정보

© 한상훈 NestJS 백엔드 마스터

about:blank와 type URL

→ type을 정하지 않은 에러: `"about:blank"`, 상태 코드 이상의 정보 없음

→ `about:blank`일 때 title: 상태 코드의 이름 (Not Found, Bad Request)

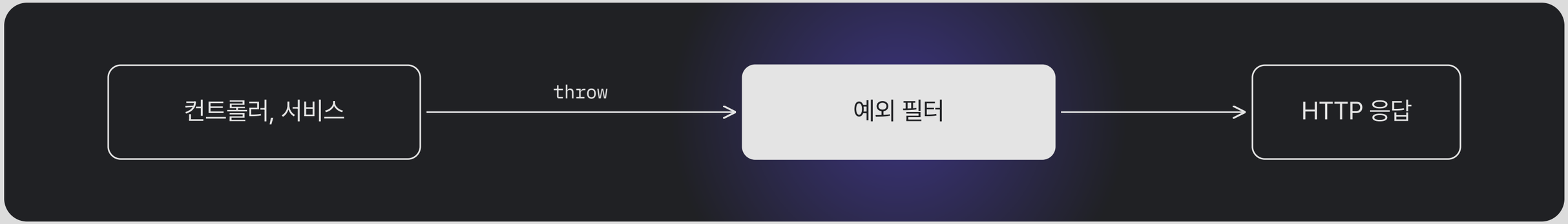
→ type URL: 바꾸지 않는 식별자, 그 주소에 문제 설명 문서를 두는 것도 가능

→ 고객 앱의 분기: type 비교

→ 서버 쪽: type별 에러 건수 집계, 늘어난 실패 종류 확인

© 한상훈 NestJS 백엔드 마스터

예외 필터



→ 예외 필터(exception filter): 던진 예외를 받아 HTTP 응답으로 바꾸는 곳

→ Nest 기본 필터: 지금까지의 message·error·statusCode 본문

→ `@Catch()`: 모든 예외를 받는 필터

→ 서버 전체에 하나: `useGlobalFilters`

© 한상훈 NestJS 백엔드 마스터

ProblemException

```ts
// apps/ticketing/src/common/problem.exception.ts
export class ProblemException extends HttpException {
  constructor(
    status: number,
    readonly type: string,
    readonly title: string,
    detail: string,
    readonly extensions: Record<string, unknown> = {},
  ) {
    super(detail, status);
  }
}

export class SeatAlreadyHeldException extends ProblemException {
  constructor(seat: string) {
    super(409, 'https://api.ticketing.example/problems/seat-already-held',
      '이미 선점된 좌석', `${seat} 좌석은 다른 고객이 선점했습니다.`, { seat });
  }
}
```

© 한상훈 NestJS 백엔드 마스터

ProblemDetailsFilter: HttpException

```
@Catch()
export class ProblemDetailsFilter implements ExceptionFilter {
  catch(exception: unknown, host: ArgumentsHost) {
    const req = host.switchToHttp().getRequest<Request>();
    const res = host.switchToHttp().getResponse<Response>();
    let status = 500;
    let body: Record<string, unknown>;
    if (exception instanceof ProblemException) {
      status = exception.getStatus();
      body = { type: exception.type, title: exception.title, status,
               detail: exception.message, ...exception.extensions };
    } else if (exception instanceof HttpException) {
      status = exception.getStatus();
      const r = exception.getResponse() as { message?: string | string[] } | string;
      const message = typeof r === 'string' ? r : r.message;
      body = { type: 'about:blank', title: STATUS_CODES[status], status };
      if (Array.isArray(message)) body.errors = message;
      else if (message) body.detail = message;
    } else { /* 다음 슬라이드 */ }
    body.instance = req.originalUrl;
    res.status(status).type('application/problem+json').json(body);
  }
}
```

© 한상훈 NestJS 백엔드 마스터

ProblemDetailsFilter: 그 밖의 예외

```
} else {
  // 내부 사정(에러 메시지, 스택): 로그에만 기록, 응답에서 제외
  this.logger.error(exception);
  body = { type: 'about:blank', title: STATUS_CODES[500], status,
           detail: '요청을 처리하지 못했습니다.' };
}
```

→ STATUS_CODES: node:http의 상태 코드 이름표

© 한상훈 NestJS 백엔드 마스터

전역 등록

```
// apps/ticketing/src/main.ts
app.useGlobalFilters(new ProblemDetailsFilter());
```

```
// concerts.service.ts
throw new SeatAlreadyHeldException(seat);
```

© 한상훈 NestJS 백엔드 마스터

필터 뒤의 응답

터미널

```
$ curl -i localhost:4011/concerts/99
HTTP/1.1 404 Not Found
Content-Type: application/problem+json; charset=utf-8
{"type":"about:blank","title":"Not Found","status":404,
 "detail":"공연 99번이 없습니다","instance":"/concerts/99"}

(검증 실패)
HTTP/1.1 400 Bad Request
{"type":"about:blank","title":"Bad Request","status":400,
 "errors":["seat must be a string","count must not be less than 1",
 "count must be an integer number"],"instance":"/performances/1/holds"}

$ curl -X PUT localhost:4011/concerts/1
{"type":"about:blank","title":"Not Found","status":404,
 "detail":"Cannot PUT /concerts/1","instance":"/concerts/1"}
```

409: 슬라이드 '이미 선점된 좌석'의 응답 그대로

© 한상훈 NestJS 백엔드 마스터

500 응답과 서버 로그

○ ○ ○ 터미널

```
$ curl -i localhost:4011/concerts/1/sales
HTTP/1.1 500 Internal Server Error
Content-Type: application/problem+json; charset=utf-8
{"type":"about:blank","title":"Internal Server Error","status":500,
 "detail":"요청을 처리하지 못했습니다.","instance":"/concerts/1/sales"}

(서버 로그)
ERROR [ProblemDetails] Error: settlement server did not respond
    at ConcertsController.sales (.../main.js:367:15)
```

→ 고객: 일반 문장 / 개발자: 로그의 원래 메시지와 스택

© 한상훈 NestJS 백엔드 마스터

응답에 싣지 않는 것

→ 가정: 모든 예외의 message를 detail에 그대로 싣는 필터

→ DB 에러 메시지: 테이블 이름, 컬럼 이름, 제약 조건 이름

→ 스택 트레이스: 파일 경로, 라이브러리 이름과 버전

→ 외부 서비스 에러: 내부 주소, 결제사 원문 응답

→ 싣는 것: 고객이 할 수 있는 일, 찾아볼 단서(instance, 요청 ID)

→ 검증 실패 errors: 문장 배열 → 필드 이름과 이유를 나눈 객체 배열도 선택지

© 한상훈 NestJS 백엔드 마스터

매진도 type 하나로

```
HTTP/1.1 409 Conflict
Content-Type: application/problem+json

{
  "type": "https://api.ticketing.example/problems/sold-out",
  "title": "매진",
  "status": 409,
  "detail": "10월 콘서트 1회차는 매진되었습니다.",
  "instance": "/performances/1/holds"
}
```

→ 45만 명에게 같은 type, 고객 앱은 매진 화면으로

→ 이 강의 코드에는 아직 없음, 예시

© 한상훈 NestJS 백엔드 마스터

앞단이 돌려준 HTML

```
HTTP/1.1 502 Bad Gateway
Content-Type: text/html

<html>
<head><title>502 Bad Gateway</title></head>
...
```

→ 로드 밸런서, 프록시, CDN의 기본 에러 페이지: HTML

→ 고객 앱: '알 수 없는 오류'

```
JSON.parse(text)
// Unexpected token '<', "<html>
// <h"... is not valid JSON
```

© 한상훈 NestJS 백엔드 마스터

본문을 읽기 전에

```
const type = res.headers.get('content-type') ?? '';
if (type.startsWith('application/problem+json')) {
  const problem = await res.json();
  return handleProblem(problem);          // type으로 분기
}
return handleByStatus(res.status);        // 상태 코드만으로
```

→ `Content-Type` 확인 → problem+json이면 type으로, 아니면 상태 코드로

→ 앞단의 에러 페이지: 설정으로 JSON 응답 가능한 곳도 있음

→ 처리 순서: 상태 코드로 큰 갈래 → problem+json이면 type으로 세분

© 한상훈 NestJS 백엔드 마스터

문장과 type

"에러 본문에는 사람이 읽을 문장과
프로그램이 읽을 type을 따로 싣습니다."